



10-152-170 Programming in Python (NWTTC)

Course Outcome Summary

COURSE INFORMATION

Alternate Title: Programming in Python

Description:

10-152-170 PROGRAMMING IN PYTHON [...ln](#) this course, students investigate the fundamentals of computer programming using the Python programming language. Students examine data types, variables, conditional statements, looping, array structures, and structured programming techniques. Upon completion of the course, students will be able to use Python to apply problem solving skills to create applications for delivery to various platforms. (Prerequisite: 10-152-140, IT:Program: Logic Intro; Corequisite: 10-804-189, Introductory Statistics)

Instructional Level: 10

Total Credits: 3

Total Hours: 72

COURSE HISTORY

Last Revision Date: 5/31/2022

COURSE COMPETENCIES

1. Assess the Python programming environment

Assessment Strategies

1.1. Project assignments and quiz

Criteria

Learners will be successful when they are able to:

- 1.1. Install the Python program, configuring appropriately
- 1.2. Install and run IDLE
- 1.3. Identify Python commands to complete a program solution
- 1.4. Write interactive Python applications which format and use numeric and string data types
- 1.5. Test Python code for accuracy during each iteration

Learning Objectives

- 1.a. Identify the Python / IDLE software components
- 1.b. Discuss variety of Text Editors
- 1.c. Demonstrate running a Python program
- 1.d. Evaluate naming conventions for variables
- 1.e. Define different data types, whitespace, literals, and constants
- 1.f. Determine proper use of variable data type, whitespace, and constants

2. Identify structures such as Lists, For Loops, and If Statements

Assessment Strategies

2.1. Project assignments and quiz

Criteria

Learners will be successful when they are able to:

- 2.1. Determine proper use of variable data type, whitespace, and constants
- 2.2. Write interactive Python applications which utilize lists
- 2.3. Fill lists with data manually, and by using the Range() function
- 2.4. Manipulate lists to change the data they contain
- 2.5. Enumerate lists to fill, retrieve, modify, or display contents
- 2.6. Write interactive Python applications which make decisions using the If structure
- 2.7. Assess when to utilize If functionality with multiple relational operators and Boolean expressions

Learning Objectives

- 2.a. Define Python lists with index positions
- 2.b. Determine how to add, remove, and change data in a list
- 2.c. Identify methods to organize and enumerate lists
- 2.d. Define the Range() function
- 2.e. Demonstrate copying lists and using slicing to work with part of a list
- 2.f. Define the basic and extended If structures
- 2.g. Explain conditional testing with relational operators and Boolean expressions
- 2.h. Determine when to use multiple conditions

3. Identify commands using dictionaries, while loops, and user input

Assessment Strategies

3.1. Project assignments and quiz

Criteria

Learners will be successful when they are able to:

- 3.1. Create Python dictionaries for storing data
- 3.2. Write interactive Python applications which utilize dictionaries
- 3.3. Manipulate dictionaries to change the data they contain
- 3.4. Enumerate dictionaries to fill, retrieve, modify, or display contents
- 3.5. Write interactive Python applications which utilize user input in various formats
- 3.6. Assess when to utilize program flow commands and flag variables

Learning Objectives

- 3.a. Define dictionaries using key-value pairs
- 3.b. Determine how to add, change, remove items from a dictionary
- 3.c. Identify methods to organize and enumerate dictionaries
- 3.d. Demonstrate retrieving user input in both string and numeric formats

- 3.e. Define the modulo operator
- 3.f. Describe program flow using flag variables and the 'break' and 'continue' commands

4. Define Python functions and modules

Assessment Strategies

- 4.1. Project assignments and quiz

Criteria

Learners will be successful when they are able to:

- 4.1. Create a Python function to solve a specific task
- 4.2. Convert a Python function into an importable module
- 4.3. Add modules and functions to streamline code
- 4.4. Increase reusability of Python code using modules and functions
- 4.5. Write a Python application that imports a user-defined function or module
- 4.6. Assess function creation using mandatory, optional, or arbitrary arguments
- 4.7. Assess function creation using parameters, defaults, and return data types
- 4.8. Write a Python application that utilizes the various features included in a custom function

Learning Objectives

- 4.a. Define Python functions and modules
- 4.b. Demonstrate creating functions and modules
- 4.c. Determine what arguments and parameters to include in a function or module
- 4.d. Explain function return values, default values, and optional and arbitrary arguments

5. Develop class objects, file input/output, and exception handling

Assessment Strategies

- 5.1. Project assignments and quiz

Criteria

Learners will be successful when they are able to:

- 5.1. Assess class needs, structure, and contents based on expected use
- 5.2. Create a Python class with appropriate features for a specific use
- 5.3. Write a Python application that imports, instantiates, and utilizes a class and its attributes
- 5.4. Include derived class features in a Python programming solution
- 5.5. Write Python code that can handles unexpected errors appropriately
- 5.6. Store data using JSON
- 5.7. Refactor code effectively for improved readability, troubleshooting, and performance

Learning Objectives

- 5.a. Define Python classes, including attributes, and the self and __init__ function
- 5.b. Demonstrate class creation and use in Python code
- 5.c. Define class derivation, inheritance, and polymorphism
- 5.d. Explain importing a class from a module
- 5.e. Demonstrate creating, reading from, and writing to, data files
- 5.f. Explain error determination and processing using try-catch and exception handling

5.g. Evaluate using JSON as an additional data storage tool

5.h. Demonstrate refactoring to clean and organize code

6. Implement data storage and manipulation functionality using the Python 'NumPy' and 'Pandas'

Assessment Strategies

6.1. Project assignments and quiz

Criteria

Learners will be successful when they are able to:

6.1. Import NumPy and Pandas into a Python application

6.2. Write a python application that utilizes SQL Server connection

6.3. Assess the data needs of a data-related customer problem

6.4. Write Python commands to retrieve appropriate data from SQL Server

6.5. Demonstrate data retrieval from SQL Server to store in arrays or data frames

6.6. Use NumPy dimensional arrays

6.7. Use Pandas data series and data frames

6.8. Modify chosen data structures to manipulate, and organize the data

6.9. Clean data for null values and improper input

Learning Objectives

6.a. Define the NumPy and Pandas Python packages

6.b. Determine how to import and employ the NumPy and Pandas packages

6.c. Create a connection to a SQL Server database

6.d. Define universal and aggregate data functions

6.e. Explore options for vectorizing bad data and handling missing data

6.f. Combine datasets into data collections

7. Implement data visualization functionality using the Python 'matplotlib' package

Assessment Strategies

7.1. Project assignment and quiz

Criteria

7.1. Import matplotlib into a Python application

7.2. Write a python application that utilizes matplotlib functionality

7.3. Assess data to choose the best matplotlib display method

7.4. Assess customer needs for output

7.5. Prepare data visualization output to suit customer needs

Learning Objectives

7.a. Define the matplotlib package

7.b. Determine how to import and employ the matplotlib package

7.c. Describe matplotlib plotting and graphing options

7.d. Describe matplotlib histograms and error calculations

7.e. Demonstrate how to customize matplotlib output